

A Real-Time Traffic Digital Twin from Public CCTV: Vision-Based Detection, Map-Anchored Localization, and Self-Calibrating Speed Estimation

Minsoo Ahn

Department of Computer Science
Stony Brook University
minsoo.ahn@stonybrook.edu

Dahyun Kwon

Department of Computer Science
Stony Brook University
dahyun.kwon@stonybrook.edu

Abstract—We present a real-time traffic digital twin that converts live public CCTV feeds into an interactive, map-anchored view of road traffic. From a single clicked camera the system streams HLS video, detects vehicles with Ultralytics YOLO26, an NMS-free end-to-end detector (with YOLOv8 retained as a legacy fallback), tracks them with a pluggable multi-object tracker (BoxMOT), and converts each vehicle’s pixel position to a geographic coordinate through a perspective homography. Because public CCTV cameras are uncalibrated and their intrinsics are unknown, accurate ground-plane mapping and speed estimation are the central challenges. We address them with three complementary mechanisms: (i) a two-stage curved transform that maps pixels along the true road centreline obtained from the national node-link road network rather than a flat plane; (ii) a five-stage speed estimator combining a physics-based outlier guard, least-squares regression over a sliding window, and per-track exponential smoothing; and (iii) a per-camera scale factor that absorbs the residual geometric scale error and is updated online via an ITS-referenced calibration loop. The system additionally provides multi-camera background monitoring, camera-level congestion clustering, and a SQLite time-series history with charting. We describe the architecture and the key algorithms, and report latency, throughput, tracking stability, and speed metrics produced by a measurement harness over live streams.

Index Terms—digital twin, intelligent transportation systems, object detection, multi-object tracking, homography, perspective transform, speed estimation, real-time visualization.

I. INTRODUCTION

Cities expose thousands of public traffic cameras, but the raw feeds are difficult to interpret at scale: an operator watching a video grid cannot read off vehicle counts, speeds, or where congestion is forming on a map. A *digital twin*—a live, geo-referenced model of the road state—turns those feeds into actionable information. The technical obstacle is that public CCTV cameras are *uncalibrated*: their height, pitch, focal length, and exact heading are unknown, so projecting an image detection onto the ground plane (and therefore measuring distance and speed) is ill-posed without per-camera calibration.

This paper describes a system that ingests Korean ITS CCTV streams and produces a real-time map-anchored twin. Our contributions, from a systems and algorithms standpoint, are:

- **Map-anchored localization.** We snap each camera onto the national node-link road network, extract the road centreline, and map pixels along the *curve* of the road in two stages, rather than assuming a flat homographic plane. This keeps vehicle markers on the road even on bends.
- **Robust speed estimation without ground truth.** A five-stage filter (duplicate skip, physics jump guard, sliding-window least-squares regression, per-track exponential smoothing with spike rejection, and scale correction) produces stable speeds despite detection jitter and tracker ID churn.
- **Per-camera scale correction.** Because absolute distance scale is unknowable from an uncalibrated camera, the system maintains a per-camera `speed_scale` that decouples geometric projection from absolute speed accuracy. An ITS-referenced online update loop with adaptive rate α_s (0.15 initially, decaying to 0.01) accumulates the factor across sessions; the current deployment has collected factors for 37 cameras (range 0.34–2.25), though multi-session operation is required for full convergence (see Section VI).
- **A complete, resilient pipeline.** Automatic lane-based calibration, manual 4-point calibration, dual-path inference (browser canvas and server-side HLS) over a shared detector, multi-camera background monitoring, congestion clustering, and a historical analytics store.

The rest of the paper is organized as follows. Section II reviews related work. Section III presents the system architecture. Section IV details the algorithms. Section V reports the measurements. Section VI discusses limitations, and Section VII concludes.

II. RELATED WORK

Object detection and tracking. Modern traffic vision builds on real-time detectors; we use YOLO26 [1] as the default detector—an NMS-free end-to-end architecture that produces more consistent per-frame bounding-box positions than NMS-based models such as YOLOv8 [2], reducing the ground-contact jitter that propagates into speed estimates. It

also simplifies the TensorRT FP16 engine graph (no NMS layer) [3], lowering inference latency and its variance; YOLOv8 is retained as a legacy fallback. For tracking we use the BoxMOT framework [4], which exposes ByteTrack [5], BoT-SORT [6], and OC-SORT [7]; appearance re-identification uses OSNet [8]. Trackers without appearance ReID suffer ID switches under occlusion, which we mitigate with a lightweight position-based ID stabilizer.

Camera-to-ground mapping. Mapping image points to a ground plane is a homography problem [9], typically solved from four correspondences with OpenCV [10]. For *uncalibrated* cameras, vanishing-point and lane-geometry cues are commonly used to recover pose. Our work combines a 4-point manual mode, a vanishing-point/lane auto-calibration, and—novel in this context—a road-centreline-following transform driven by an external road-network dataset [11].

ITS dashboards and data. National ITS portals [12] publish CCTV streams and aggregated segment speeds, but typically as separate views. We *fuse* the segment speed back into the pipeline as a calibration signal. Congestion grouping uses density-based clustering [13] and convex hulls [14]; service levels follow the Highway Capacity Manual [15]. The visualization uses deck.gl [16] over MapLibre [17], served by FastAPI [18].

III. SYSTEM DESIGN AND ARCHITECTURE

The backend (FastAPI/Uvicorn) and a React/deck.gl frontend communicate over REST and WebSocket. Two inference paths can produce per-frame analytics and *share a single detector*; a connection counter coordinates them so that the multi-object tracker is never driven by two interleaved frame streams (Fig. 1). A two-lock scheme inside the detector (`_state_lock` for tracker state, `_lock` for GPU inference) prevents data races between thread-pool threads, and module-level write locks in `main.py` and `roi_manager.py` atomize all JSON configuration read-modify-write operations.

Browser path (/ws/detect). The frontend plays the HLS stream through a CORS proxy, captures canvas frames (JPEG, ≤ 640 px, ≈ 33 ms interval, at most two in flight), and sends them to the backend, which runs detection, tracking, localization, and analytics, then returns an annotated frame and broadcasts the analytics JSON to all clients.

Server path (live_loop). The backend can also read the HLS stream directly with OpenCV/FFmpeg and run the same pipeline. It is gated twice: it skips tracking while a browser detection client is active (to protect the shared tracker), and it skips all GPU work when no one is watching (the frontend reports tab visibility). Before opening any HLS URL the detector pre-resolves HTTP 302 redirects (`_resolve_hls_url`), so FFmpeg receives a direct endpoint and does not silently fail on delivery-layer redirect chains.

Background services. Periodic loops refresh expiring HLS tokens (30 min), poll the ITS segment speed for self-calibration (5 min), and snapshot all monitored cameras into a SQLite history while recomputing congestion clusters (30 s). To prevent the FOV polygon from flickering during auto-calibration, a

slow EMA filter stabilizes near/far distance and road-width estimates, holding the initial accepted value fixed until sufficient samples accumulate. ROI edits trigger an immediate GPS ring recomputation and broadcast to keep the map view synchronized. The resulting map view is shown in Fig. 2.

IV. IMPLEMENTATION

A. Detection and Tracking

The detector loads the fastest available backend—TensorRT FP16 engine, then ONNX Runtime, then PyTorch—and exports `.pt`→`.engine` on first run when a GPU is present. A tracker *tier* is chosen by GPU memory: ByteTrack (CPU-only, no CUDA), OC-SORT (low VRAM ≥ 3.5 GB, occlusion-robust, no ReID), or BoT-SORT/DeepOC-SORT with OSNet appearance ReID on larger GPUs (≥ 6 GB and ≥ 10 GB respectively). For the non-ReID tiers we add two robustness layers: `_dedup_tracks` merges duplicate boxes (IoU > 0.3 or centre distance < 40 px, keeping the older ID), and an `IDStabilizer` re-binds a vehicle’s previous ID after a brief miss using nearest-centre matching (≤ 80 px) with a two-pass scheme and stale-mapping purge so display IDs stay stable and compact. Fig. 3 shows the detection and tracking overlay.

B. Map-Anchored Localization

On camera switch we query the national node-link network: we snap the camera position perpendicularly onto the nearest road polyline, extract ± 150 m of centreline, and—because each direction is stored as a separate one-way link—we locate the reverse carriageway and average the two snaps (when 2–40 m apart) to recover the true road centre. The chosen road’s speed limit, lane count, and width (lanes $\times 2 \times$ lane-width) are derived from NodeLink and attached to the camera.

Given the centreline, the *two-stage curved transform* maps a pixel (u, v) to GPS along the road (Algorithm 1). The metric homography H_m is derived from the same four image–GPS correspondences as the GPS homography, but maps pixel coordinates to local ENU metres rather than geographic coordinates. Stage one maps the pixel to local metres via H_m ; stage two decomposes the displacement from the snap point into along-road and lateral components, advances that arc length along the centreline, interpolates the on-road position, and re-applies the lateral offset perpendicular to the *local* road bearing. This follows real road curvature, unlike a single flat homography.

When no manual calibration exists, an automatic procedure estimates a homography from one frame: Canny edges in the lower image, Hough line filtering, multi-row sampling of left/right road edges, least-squares edge fits, vanishing-point intersection, a pitch/height estimate from a pinhole model, and a trapezoid-to-GPS homography. Direction (which way the camera looks along the road) is resolved by matching the *image* road-curve sign against the *map* curve sign, falling back to a vanishing-point heading test. A manual 4-point mode remains available for maximum accuracy. Fig. 4 shows an automatic calibration result.

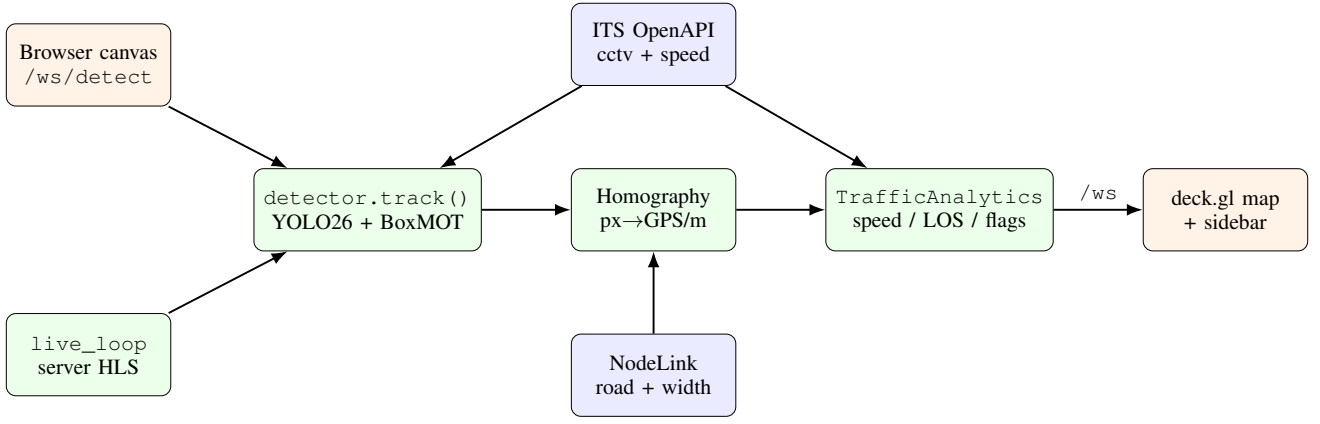


Fig. 1. System architecture. The browser path (/ws/detect) and the server path (live_loop) share one detector; a counter prevents concurrent tracker calls. External data (ITS CCTV/speed, NodeLink) feeds localization and self-calibration.

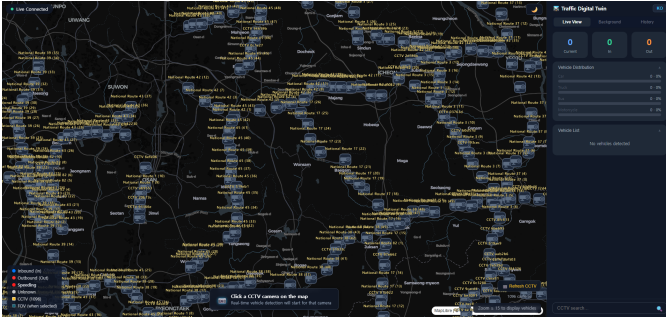


Fig. 2. Map view: public CCTV cameras as status-coloured icons on the vector map; the selected camera shows its field-of-view polygon and the located vehicles.



Fig. 3. Detection and tracking on a live CCTV frame: bounding boxes, vehicle class, and stabilized track IDs.

The primary auto-calibration path is a road-model *pose solver* (`camera_pose.solve_pose`): the sampled lane edges, vanishing point, and NodeLink road width are fitted to a four-parameter pinhole-camera model (H , pitch, yaw, lateral offset x_0) via `scipy.optimize.least_squares` (soft-L1 loss). A per-row reprojection residual gates the output

Algorithm 1 Two-stage curved pixel→GPS transform

Require: pixel (u, v) ; metric homography H_m ; snap point p_m (ENU m); road bearing b at snap; centreline $road_pts$; $snap_along$; direction sign s

- 1: $(x_m, y_m) \leftarrow H_m \cdot (u, v)$
- 2: $(dx, dy) \leftarrow (x_m, y_m) - snap_m \quad \triangleright$ ENU from snap
- 3: $d_{||} \leftarrow dx \sin b + dy \cos b; \quad d_{\perp} \leftarrow dx \cos b - dy \sin b$
- 4: $arc \leftarrow snap_along + s \cdot \max(0, d_{||})$
- 5: $(lat, lon) \leftarrow \text{INTERPCENTERLINE}(arc)$
- 6: $b_{\ell} \leftarrow \text{ROADBEARINGAT}(arc)$
- 7: **apply** $d_{\perp}$ perpendicular to b_{ℓ} **return** (lat, lon)

(<8 px); on success the solved pose is converted to four image-GPS corner pairs and used to build both homographies. The pose is then persisted per camera to `camera_pose.json` so each session refines the prior, with a cold-start fallback chain (saved prior $\rightarrow$ vehicle-bbox rough pose $\rightarrow$ GPS-grid). A complementary `vehicle_calib.json` accumulates a per-camera linear pixel-to-distance model ($d = B \cdot y_{px} + C$, implied vanishing-point $y_v = -C/B$) fitted from observed vehicle bounding-box heights; it is restored on session start and updated online whenever sufficient samples accumulate, providing an additional distance scale that cross-checks the geometric homography. The trapezoid heuristic above serves only as the fallback when the residual exceeds the quality gate or lane detection fails.

After GPS coordinates are assigned, a two-step post-processing pass reduces visual noise. Per-track EMA smoothing eliminates lateral jitter while preserving along-road motion; a jump threshold resets the filter after occlusion gaps to handle track re-appearances correctly. A lane-offset pass then laterally separates inbound and outbound vehicles perpendicular to the local road bearing, placing each direction on its correct side of the road centreline for Korean right-hand traffic.

C. Speed Estimation

Speed is distance over time, and both terms are error-prone: homography distance is noisy at the far field, and HLS buffering

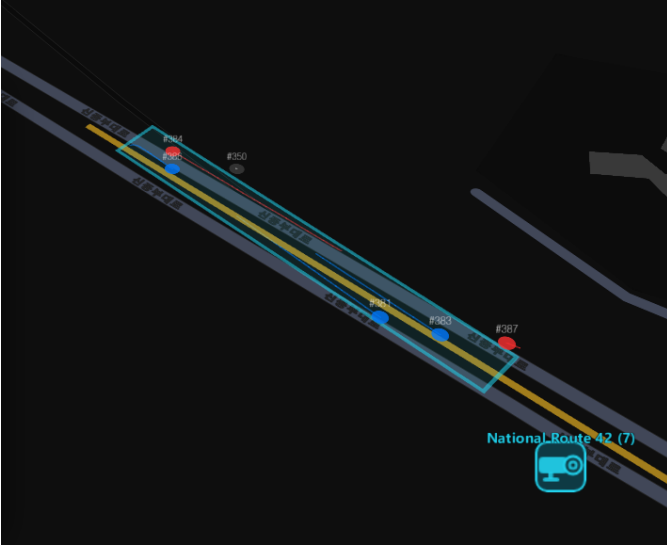


Fig. 4. Map view showing the estimated field-of-view footprint after automatic calibration: the cyan corridor follows the road centreline extracted from the NodeLink dataset.

Algorithm 2 Per-track speed estimation (`analytics._speed`)

Require: window W , prior EMA v_e , $\sigma \leftarrow \text{speed_scale}$,
 $\kappa \leftarrow \text{depth_corr}(y_{\text{bbox}})$

- 1: **if** duplicate position **then** skip ▷ dedup
- 2: **if** $\text{step} > (V_{\text{max}}/3.6\sigma) dt \cdot 3.0$ **then** clear W ▷ teleport reset
- 3: **else if** $\text{step} > (V_{\text{max}}/3.6\sigma) dt \cdot 1.5$ **then** drop sample, keep W ▷ jump guard
- 4: **if** $dt > 2\text{ s}$ **then** clear W ▷ track gap
- 5: fit $\hat{v}$ by OLS on W (0.7 s); **if** $\text{disp}(W) < 0.5\text{ m}$ **then** $\hat{v} \leftarrow 0$ ▷ OLS
- 6: $v_e \leftarrow (1-\alpha)v_e + \alpha\hat{v}\kappa\sigma$, $\alpha=0.35$; reject spike; decay on stop ▷ EMA
- 7: **output** v_e ; $\text{is_speeding} \leftarrow \text{reliable} \wedge v_e > 1.10 \cdot \text{limit}$ ▷ output

makes naive frame timing wrong. We use a monotonic clock derived from stream presentation timestamps (falling back to wall-clock), and the five-stage estimator of Algorithm 2 with the constants in Table I.

Two additional reliability mechanisms reduce systematic bias in the speed estimate. First, a depth-varying correction factor κ is computed from a per-camera vehicle apparent-size model (a linear fit of inverse apparent bbox width versus vertical pixel position), compensating for the depth-dependent projection error of the homography without introducing frame-to-frame position discontinuities. Second, vehicles whose GPS position lies more than 100m from the camera snap point (`SPEED_TRUST_MAX_DEPTH_M`) are flagged `speed_reliable=False`: their speed is still displayed, but excluded from the frame average, the ITS calibration input, and speeding alerts, preventing far-field pixel noise from biasing aggregate

TABLE I
KEY SPEED/ANALYTICS CONSTANTS (`CONFIG.PY`).

Parameter	Value
Speed window	0.7 s ($W=\text{round}(0.7s \times \text{FPS})$)
EMA α	0.35
Jitter threshold	0.5 m
Min speed (else 0)	5 km/h
Max plausible V_{max}	180 km/h
GC grace	30 frames
Bottleneck dwell	150 frames ($\approx 5\text{ s}$)
Parked dwell	300 frames ($\approx 10\text{ s}$)
Speeding threshold	$v_e > 1.10 \times \text{limit}$
LOS thresholds A–E	$\leq 3, 6, 9, 12, 15$ (else F)
Background busy	> 6 vehicles
Background congested	> 14 vehicles
Cluster ε	500 m (haversine)

statistics.

D. Per-Camera Scale Correction

The absolute distance scale of an uncalibrated camera is unknowable from geometry alone, so a residual scale error persists after geometric calibration. The system maintains a per-camera `speed_scale` factor that multiplies the geometric speed estimate (Algorithm 2, line 6) and is updated online using the ITS official segment speed as a soft reference. The ITS signal carries inherent uncertainty—aggregated over $\approx 1\text{ km}$ segments at 5-minute intervals, the reported direction may not match the camera’s field of view—so the loop adapts slowly:

$$\text{target} = \text{scale} \cdot \frac{v_{\text{ITS}}}{v_{\text{ours}}}, \quad \text{scale} \leftarrow (1-\alpha_s) \text{scale} + \alpha_s \text{target}, \quad (1)$$

with an adaptive α_s schedule and a per-update $\pm 10\%$ step clamp in addition to the absolute $[0.3, 5.0]$ bounds. For cameras without a previously saved factor (first session), α_s decreases from 0.15 (updates 1–2) through 0.05 (updates 3–4) to 0.01 thereafter, enabling fast initial convergence while damping later oscillation. For cameras whose factor was restored from `speed_scale.json`, α_s is fixed at 0.01 to preserve the accumulated calibration. The calibration uses a 10-minute rolling window of continuous, per-frame speed observations, requiring at least 50 frames with valid vehicle detections to trigger the correction, and a coefficient-of-variation guard (≤ 0.4) to skip volatile traffic. The factor persists per camera and accumulates across sessions without resetting the geometric calibration (Section VI).

E. Aggregation, Congestion, and History

Direction is primarily determined by projecting each vehicle’s inter-frame displacement onto the road bearing axis with an EMA deadzone filter; a horizontal LineZone crossing serves as a fallback when road bearing is unavailable, and also provides In/Out counts. Dwell counting flags bottlenecks and parked vehicles (the latter remembered by pixel position so they survive ID changes), and a Level-of-Service grade is assigned from the active vehicle count. Background monitoring polls additional cameras every 8 s with detection only; cameras with



Fig. 5. Camera-level congestion: monitored cameras are recoloured (normal/busy/congested) and adjacent congested cameras are grouped into a severity-shaded region.

more than 6 vehicles are marked *busy* and those with more than 14 are marked *congested*; such cameras are then clustered with a haversine density-based scan ($\epsilon=500$ m) and outlined with a convex hull. A WAL-mode SQLite store records 30 s snapshots (14-day retention) and serves bucketed time series, peak-time detection, and CSV export. Fig. 5 shows the congestion overlay.

F. Visualization

The deck.gl map layers congestion polygons, vehicle trails, the FOV polygon, status-coloured camera icons, and vehicle markers. The FOV polygon uses one of three strategies in priority order: (1) the manual calibration ring (calibGpsRing), the exact four GPS corners entered by the user; (2) a curved polygon computed by walking the road centreline and laterally offsetting by half the road width (computeRoadCorridorPolygon), so the footprint follows real road curvature rather than a flat projection; or (3) a straight-sided rectangle derived from near/far distance and road width (computeCalibPolygon), falling back to a default 70° trapezoid (computeFovPolygon) when uncalibrated.

The sidebar’s vehicle list provides *direction-filter tabs* (All / Inbound / Outbound) that filter the displayed rows by the movement-vector-based direction; the per-direction vehicle count is shown in each tab badge, and the min/avg/max speed summary is recomputed from the filtered set. Two collapsible cards (Auto Calibration Estimate, ITS Speed Comparison) include an info button that opens a modal explanation of each metric, adapting to the selected map theme.

TABLE II
INFERENCE BACKEND COMPARISON (OC-SORT TRACKER FIXED, 1,000 FRAMES, RTX 4070 LAPTOP). TRACK STAGE INCLUDES YOLO + BOXMOT END-TO-END.

Backend	Mean (ms)	p95 (ms)	FPS
TensorRT FP16	6.7	8.7	147.6
PyTorch (CUDA)	8.4	11.6	117.6
ONNX Runtime	13.9	16.8	71.5

TABLE III
TRACKER COMPARISON (TENSORRT FP16 BACKEND FIXED, 1,000 FRAMES). BoT-SORT OVERHEAD IS FROM OSNET APPEARANCE REID; BYTETRACK SHOWS MORE ID CREATION EVENTS DUE TO THE ABSENCE OF RE-IDENTIFICATION.

Tracker	Mean (ms)	p95 (ms)	FPS	IDs	ID ev.	Life (fr.)
ByteTrack	6.6	7.8	150.9	27	286	35.6
OC-SORT	6.6	8.9	150.2	8	18	80.0
BoT-SORT	10.7	18.5	93.4	5	16	132.0

IDs = unique track IDs; ID ev. = ID creation events; Life = mean lifetime.

V. EVALUATION

Measurements are produced from the real pipeline in two ways. While the system runs (make dev), an in-process collector records per-frame stage latency, tracking, speed, and detection statistics and flushes eval_*.csv and eval_summary.json every 30 s (also on demand via GET /eval/report); an offline harness (backend/evaluate.py) reproduces the same metrics over a live HLS source for controlled benchmarks. The system auto-selects the inference backend (TensorRT → ONNX Runtime → PyTorch) and tracker tier (ByteTrack / OC-SORT / BoT-SORT / DeepOC-SORT by GPU VRAM).

Hardware. All timing figures are from a single machine: NVIDIA GeForce RTX 4070 Laptop GPU (8 GB VRAM, compute capability 8.9), driver 610.47, TensorRT 10.16, ONNX Runtime 1.25, Windows 11. Absolute latency and FPS are hardware-specific; the relative ordering across tiers and backends is the transferable result.

Run conditions. Model YOLO26m, source: National Route 6 Namyangju Dongmakgol Intersection (live ITS HLS, nighttime), 1,000 frames per run (20-frame warmup excluded). Backend and tracker are varied across runs as shown in Tables II and III; all other conditions are held constant. Each table is an independent measurement run, so the shared TensorRT + OC-SORT operating point differs slightly across Tables II–III, and the tracking-stability counts vary with the traffic present at each capture time. Speed and vehicle-count figures in Table IV are from the TensorRT + OC-SORT run and reflect nighttime traffic at this specific location; they are included to confirm the pipeline produces plausible output, not as ground-truth benchmarks.

TensorRT FP16 achieves the lowest latency (6.7 ms mean), $2.1\times$ faster than ONNX Runtime and $1.25\times$ faster than PyTorch on the same GPU. ByteTrack and OC-SORT are iso-latency (≈ 6.6 ms), but OC-SORT’s occlusion-centric cost function

TABLE IV
TRACKING STABILITY, SPEED DISTRIBUTION, AND DETECTION
BREAKDOWN (TENSORRT + OC-SORT RUN, 1,000 FRAMES, NIGHTTIME).

Metric	Value
Frames processed	1,000
Unique tracks	12
Track creation events (re-entries + re-acquisitions)	58 ($4.83 \times$ unique tracks)
Mean track lifetime (frames)	124.2
Median track lifetime (frames)	131.5
Max track lifetime (frames)	229
Vehicle speed – mean (km/h)	90.9
Vehicle speed – median (km/h)	78.2
Vehicle speed – p95 (km/h)	174.2 [†]
% moving observations	79.6 %
Total speed observations	1,186 of 1,490 detections [‡]
Detections – car	1,424
Detections – truck	66

[†]Far-field outliers; median 78.2 km/h is representative.

[‡]Speed requires $\geq W$ track-history frames; single-frame detections excluded.

TABLE V
DISTRIBUTION OF LEARNED `SPEED_SCALE` FACTORS ACROSS 37
CAMERAS (`SPEED_SCALE.JSON`). VALUES BELOW 1.0 INDICATE
GEOMETRIC UNDERESTIMATION OF DISPLACEMENT (COMMON WHEN THE
CAMERA AXIS IS OBLIQUE TO THE ROAD); VALUES ABOVE 1.0 INDICATE
OVERESTIMATION. ALL FACTORS ARE UNCONVERGED AT THIS SNAPSHOT.

Range	Cameras	Share
< 0.5	4	10.8 %
0.5–0.8	10	27.0 %
0.8–1.2	19	51.4 %
> 1.2	4	10.8 %
Min / Max: 0.34 / 2.25		
Mean / Median: 0.90 / 0.85		

reduces ID creation events from 286 to 18 and more than doubles mean track lifetime ($35.6 \rightarrow 80.0$ frames); BoT-SORT adds OSNet ReID, further reducing ID events to 16 and improving lifetime to 132.0 frames at the cost of $1.6 \times$ higher latency (10.7 ms) and a wider p95 (18.5 ms vs 8.9 ms).

Per-camera scale calibration. Because no per-vehicle ground truth exists for public CCTV, the system maintains a per-camera `speed_scale` updated online via the ITS-referenced loop. Table V summarises the factors accumulated across all 37 cameras whose streams have been monitored (`speed_scale.json`); none had converged at the time of writing (see Section VI). The majority (51 %) fall within ± 20 % of 1.0, confirming that the geometric calibration is roughly correct for most cameras; the long left tail (38 % below 0.8) reflects cameras with steep pitch or far-field observation zones that compress apparent displacement, while the small right tail (11 % above 1.2) corresponds to wide-angle or low-mounted cameras that exaggerate it.

VI. DISCUSSION AND LIMITATIONS

The homography is accurate inside the calibration quadrilateral and extrapolates with growing error toward the far field; we mitigate this with the dual-matrix calibration, the regression window, and lane-based auto-calibration, but a residual error remains on poorly observed cameras. Automatic calibration assumes a focal length of $1.2 \times$ image height (unknown intrinsics), giving a ± 15 – 20 % scale uncertainty per FoV mismatch. It also fails when lane markings are not visible (night, rain, dense traffic), falling back to an approximate grid. Road width is inferred from lane count and assumed bidirectional, so one-way roads are over-wide. The nearest node-link road is occasionally the wrong one (e.g., a camera near both a national route and an expressway), which displaces the FOV polygon and vehicle GPS; the CCTV-name road-hint reduces but does not eliminate this. Finally, congestion is clustered at the camera level because background cameras expose only counts, not per-vehicle speeds.

The ITS-based self-calibration loop uses an adaptive learning rate (0.15 on the first two updates, decaying to 0.01 in later updates, every 5 min) to treat the ITS segment speed as an imprecise soft reference, since ITS figures reflect 1-km road-segment averages that may not align with the camera’s exact observation zone or direction. The adaptive schedule accelerates initial convergence for new cameras while preserving accumulated corrections for returning ones, but multi-session operation is still required for the factor to fully stabilize; none of the 37 accumulated cameras had converged at time of evaluation. The scale factors therefore capture the geometric projection bias present at each camera and should be interpreted as a running estimate rather than a definitive calibration.

Because the system derives all geographic positions from map geometry rather than on-vehicle GPS receivers, localization accuracy is bounded by the quality of the NodeLink road database and the precision of the perspective projection; errors in map alignment or road-shape approximation propagate directly into vehicle GPS coordinates. Additionally, a camera’s effective field of view is strongly shaped by its mounting angle: a steeply pitched installation sees a narrow, elongated strip of road, producing a thin FOV polygon on the map and limiting the usable observation window even after geometric calibration.

A structural limitation is the system’s dependence on national ITS infrastructure: camera streams and segment speeds are fetched from a government-operated open API [12] whose availability and authentication policy are outside the operator’s control and may be restricted or discontinued. Camera placement and viewing angle are fixed by road authorities and cannot be adjusted; some installations are positioned too far or too close to capture a useful road segment, and extreme viewpoints cannot be recovered through calibration. Adverse conditions—low illumination, rain, lens contamination, and hardware faults—can suppress the visual signal entirely, leaving the pipeline with no basis for detection or calibration. These constraints are inherent to re-using existing infrastructure without modification.

VII. CONCLUSION

We built a real-time traffic digital twin that localizes vehicles on a map from uncalibrated public CCTV and estimates their speed without ground truth. The key ideas are a road-centreline-following two-stage transform, a robust five-stage speed estimator, and a per-camera scale-correction framework with an ITS-referenced online update loop, wrapped in a resilient dual-path pipeline with congestion analytics and a historical store. The automatic calibration pipeline—geometric pose solver plus the accumulating scale factor—substantially reduces the per-camera manual setup required to deploy across many cameras, while the system remains operational even before the ITS loop converges.

A distinguishing property is that the system requires *no GPS instrumentation on vehicles*: conventional traffic monitoring relies on embedded GPS loggers or dedicated roadside sensors, both of which demand significant infrastructure investment. By re-purposing cameras that road authorities already operate, the system tracks multiple vehicle classes simultaneously across multiple views with no additional hardware cost. As public CCTV coverage continues to expand and object detectors advance, this class of vision-only traffic twin becomes increasingly practical at scale. With wider network integration, the same localization principle—positioning objects through perspective projection rather than satellite signal—could extend to GPS-denied environments such as tunnels, underground parking, and indoor logistics facilities; the per-camera track-ID scheme further enables cross-camera vehicle re-identification for a range of fleet management and security applications.

The source code is publicly available at <https://github.com/MinsooAhn02/DigitalTwin>.

CONTRIBUTION AND NOVELTY

This work distinguishes itself from related systems along four axes.

Versus flat-homography localization. Existing CCTV-based vehicle trackers apply a single perspective homography that treats the road as a flat plane; on curved roads this displaces markers by several metres. We introduce a two-stage transform that advances arc-length along the true road centreline extracted from the national NodeLink dataset, keeping positions geometrically correct on bends without any additional calibration target.

Versus ITS dashboards. National ITS portals (e.g., Korea’s ITS OpenAPI) publish CCTV streams and segment speeds as separate, human-facing views with no per-vehicle analytics. We *fuse* the ITS segment speed as a machine-readable calibration target in a closed online loop that converges a per-camera scale factor without any ground-truth instrumentation on vehicles or road.

Versus dedicated sensor systems. Loop detectors, radar units, and on-vehicle GPS loggers require roadside installation or vehicle participation. Our system re-purposes public CCTV cameras that road authorities already operate, adding no hardware to the road or to vehicles, yet tracking multiple vehicle classes simultaneously across multiple views.

Versus open-source YOLO/BoxMOT pipelines. Generic multi-object tracking pipelines produce bounding boxes and track IDs but lack geographic anchoring, calibrated speed estimation, or support for uncalibrated cameras. To our knowledge this is the first publicly available end-to-end implementation (<https://github.com/MinsooAhn02/DigitalTwin>) that integrates NMS-free YOLO26 detection with TensorRT FP16 acceleration, road-centreline-anchored localization, a five-stage ITS-calibrated speed estimator, and interactive deck.gl map visualization—all operating on uncalibrated public infrastructure cameras with no additional hardware.

The two authors co-designed the overall architecture and integrated their parts; the split below reflects primary ownership.

Minsoo Ahn — Detection/Tracking, Analytics, and System Infrastructure

- *BoxMOT integration and tracker-tier architecture*: the baseline ran YOLO frame-by-frame with no identity tracking. Added multi-object tracking via BoxMOT [4] and a VRAM-based tier system that automatically selects ByteTrack [5], OC-SORT [7], BoT-SORT [6], or DeepOC-SORT by available GPU memory, scaling from a CPU-only machine to a high-end workstation without code changes.
- *TensorRT inference backend*: the original inference path was PyTorch-only. Added TensorRT [3] FP16 engine export and an auto-selection chain (engine → ONNX Runtime → PyTorch), raising throughput and reducing latency variance.
- *YOLO26 upgrade*: migrated the default detector from YOLOv8 [2] to YOLO26 [1], an NMS-free end-to-end architecture, combined with TensorRT to form the current production pipeline. Raised `YOLO_CONF` from 0.25 to 0.30 since post-NMS filtering is no longer available at the detection level.
- *CCTV-based five-stage speed estimator*: replaced the naive distance/time speed approximation with the filter of Algorithm 2 (physics jump guard, sliding-window OLS regression, EMA smoothing with spike rejection, PTS-derived time axis), addressing the specific challenges of CCTV-based measurement. Added a per-vehicle `speed_reliable` flag: vehicles whose GPS position lies more than 100m from the camera snap point are excluded from the frame average and ITS calibration samples (far-field pixel noise amplification), and `is_speeding` is suppressed for unreliable vehicles.
- *Dwell counting — bottleneck and parked-vehicle detection*: added dwell-frame counting and pixel-position memory to distinguish stopped-in-traffic from genuinely parked vehicles, surviving tracker ID cycling.
- *Backend infrastructure and thread-safety*: built a FastAPI dual-path server with shared-detector coordination, congestion clustering, SQLite history store, and background multi-camera monitoring. To prevent data races across the dual-path inference pipeline, introduced a four-lock concurrency model (`_state_lock`, `_`

`json_file_lock`, `roi_manager._write_lock`, `_frame_count_lock`) with all GPU-inference lock acquisitions routed through `asyncio.to_thread`. Hardened HLS stream connection in `open_video_source`: after `isOpened()` succeeds, the function loops `read()` for up to 3 seconds to confirm a frame actually arrives—catching token-expired or 404-segment failures that previously caused silent infinite-retry loops. `VideoStream.reconnect()` was moved to `asyncio.to_thread` so the 3-second blocking validation no longer stalls the event loop.

- *Movement-vector direction classification*: replaced the LineZone-only direction scheme with a continuous movement-based classifier that projects each vehicle’s inter-frame displacement onto the road bearing axis with an EMA deadzone filter; LineZone crossing is retained as a fallback when road bearing is unavailable.
- *GPS position post-processing pipeline*: added per-track EMA position smoothing to remove lateral jitter, and a lane-offset pass that laterally separates inbound and outbound vehicles by the local road bearing perpendicular, producing physically correct lane placement for Korean right-hand traffic.
- *FOV polygon stabilization and ROI broadcast*: introduced a slow EMA filter on auto-calibrated FOV parameters (near/far distance, road width) to prevent polygon flicker; ROI edits now trigger an immediate GPS ring recomputation and live broadcast.

Dahyun Kwon — Localization, Calibration, and Map Visualization

- *In/Out vehicle counting baseline*: the original system had no reliable counting or direction assignment. Built the initial `VehicleTracker` with a horizontal `LineZone` that yields per-frame In/Out counts and per-vehicle direction, providing the foundation for directional traffic analysis.
- *Auto-calibration pipeline*: evolved calibration from a basic trapezoid heuristic to a full road-model pose solver—a four-parameter pinhole fit (H , pitch, yaw, lateral offset x_0) via `scipy.optimize.least_squares` (soft-L1) that persists per camera and refines across sessions. A cold-start fallback chain covers edge cases (saved prior $\rightarrow$ vehicle-bbox pose $\rightarrow$ GPS-grid).
- *Per-camera scale correction loop*: designed and implemented an online ITS-referenced update loop that compares the measured rolling average against the official ITS segment speed and accumulates a per-camera `speed_scale`, with a CV guard to skip volatile traffic, an adaptive α schedule (0.15 initially $\rightarrow$ 0.01 steady-state) for fast initial convergence, and a $\pm 10\%$ per-update clamp to guard against corrupt ITS samples. The factor persists across sessions and accumulates toward an ITS-validated correction over extended operation.
- *NodeLink road network integration*: identified the MOCT NodeLink dataset [11] as the geometric anchor and drove

its full integration: road snapping, centreline extraction, bidirectional carriageway fix, and road-width derivation from lane count.

- *Two-stage curved pixel $\rightarrow$ GPS transform*: the flat homography places vehicles incorrectly on curved roads. Added arc-length interpolation along the road centreline that keeps GPS positions correct on curves (Algorithm 1).
- *Automatic ROI and single-click camera setup*: extended automatic calibration to include ROI estimation, so clicking a camera triggers calibration, ROI, and road-snapping with no manual step.
- *Camera geometry utilization*: the pose solver explicitly recovers camera height and pitch from lane geometry, making the system self-aware of its viewing geometry without any pre-installed metadata.
- *Map visualization and calibration UI*: added `deck.gl` FOV polygon strategies (ROI GPS ring / manual calibration ring / straight-sided rectangle / default trapezoid), calibration overlay with NodeLink node snapping, and the calibration tab in the CCTV player.

ACKNOWLEDGMENT

The authors thank Prof. Jihoon Ryoo of SUNY Korea for guidance and feedback throughout CSE327 Computer Vision. This work used the Korea Intelligent Transport Systems (ITS) Open API and the Ministry of Land, Infrastructure and Transport (MOCT) NodeLink road network dataset, both provided by the Republic of Korea.

REFERENCES

- [1] Ultralytics, “YOLO26: Key architectural enhancements and performance benchmarking for real-time object detection,” <https://arxiv.org/abs/2509.25164>, 2025, arXiv:2509.25164.
- [2] G. Jocher, A. Chaurasia, and J. Qiu, “Ultralytics YOLOv8,” <https://github.com/ultralytics/ultralytics>, 2023, open-source object detection and tracking framework.
- [3] NVIDIA, “TensorRT: High-performance deep learning inference optimizer and runtime,” <https://developer.nvidia.com/tensorrt>, 2023.
- [4] M. Broström, “BoxMOT: Pluggable SOTA multi-object tracking modules,” <https://github.com/mikel-brostrom/boxmot>, 2023.
- [5] Y. Zhang, P. Sun, Y. Jiang, D. Yu, F. Weng, Z. Yuan, P. Luo, W. Liu, and X. Wang, “ByteTrack: Multi-object tracking by associating every detection box,” in *Proc. European Conf. on Computer Vision (ECCV)*, 2022, pp. 1–21.
- [6] N. Aharon, R. Orfaig, and B.-Z. Bobrovsky, “BoT-SORT: Robust associations multi-pedestrian tracking,” *arXiv preprint arXiv:2206.14651*, 2022.
- [7] J. Cao, J. Pang, X. Weng, R. Khirodkar, and K. Kitani, “Observation-centric SORT: Rethinking SORT for robust multi-object tracking,” in *Proc. IEEE/CVF Conf. on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [8] K. Zhou, Y. Yang, A. Cavallaro, and T. Xiang, “Omni-scale feature learning for person re-identification,” *Proc. IEEE/CVF Int. Conf. on Computer Vision (ICCV)*, 2019.
- [9] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge University Press, 2004.
- [10] G. Bradski, “The OpenCV library,” *Dr. Dobbs’s Journal of Software Tools*, 2000.
- [11] Korea Transport Database (KTDB) / Ministry of Land, Infrastructure and Transport, “Standard node-link (MOCT) road network dataset,” <https://www.its.go.kr/nodelink>, 2023.
- [12] National Transport Information Center (ITS), Republic of Korea, “ITS open API: Real-time CCTV and traffic information,” <https://openapi.its.go.kr>, 2023.

- [13] M. Ester, H.-P. Kriegel, J. Sander, and X. Xu, "A density-based algorithm for discovering clusters in large spatial databases with noise," in *Proc. 2nd Int. Conf. on Knowledge Discovery and Data Mining (KDD)*, 1996, pp. 226–231.
- [14] A. M. Andrew, "Another efficient algorithm for convex hulls in two dimensions," *Information Processing Letters*, vol. 9, no. 5, pp. 216–219, 1979.
- [15] Transportation Research Board, "Highway capacity manual (HCM)," *National Research Council, Washington, D.C.*, 2016.
- [16] vis.gl / Open Visualization, "deck.gl: WebGL2-powered geospatial visualization framework," <https://deck.gl>, 2023.
- [17] MapLibre Contributors, "MapLibre GL JS," <https://maplibre.org>, 2023.
- [18] S. Ramírez, "FastAPI," <https://fastapi.tiangolo.com>, 2018.